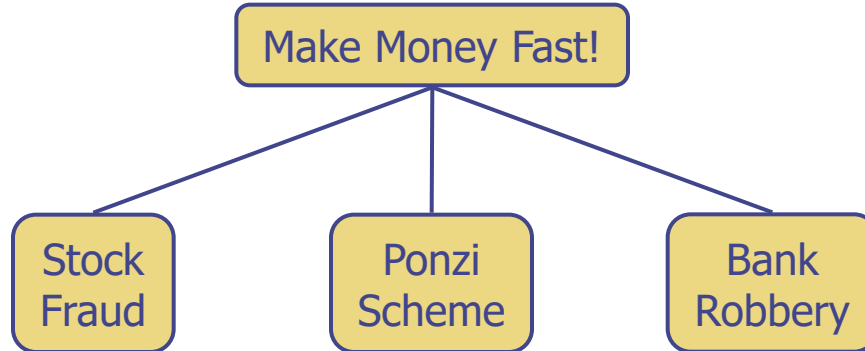
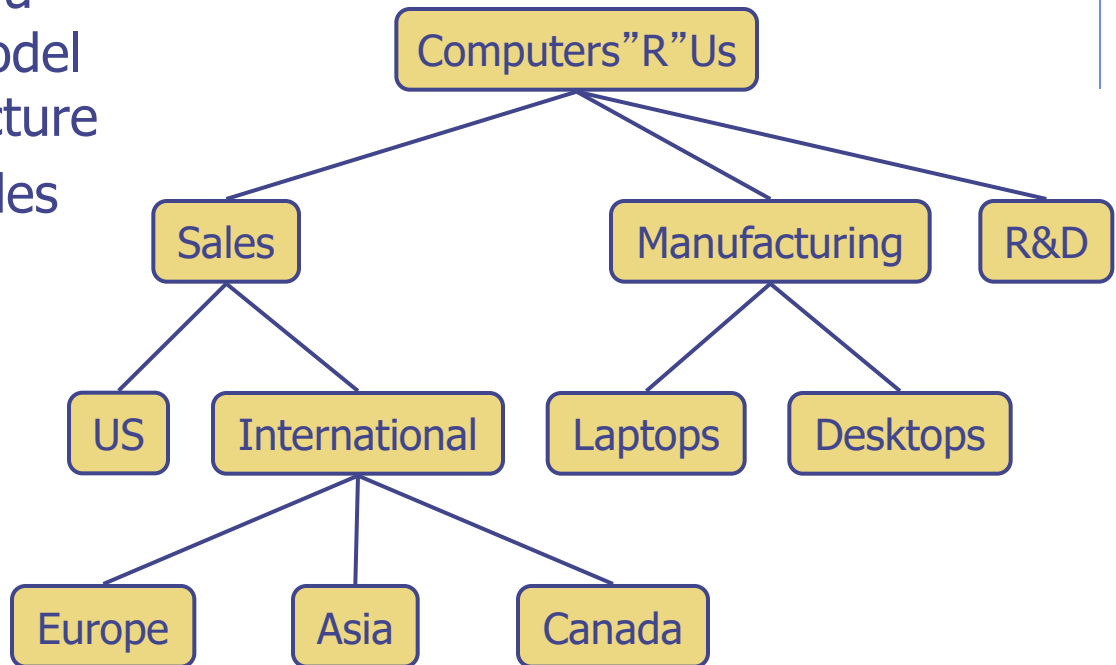


Trees

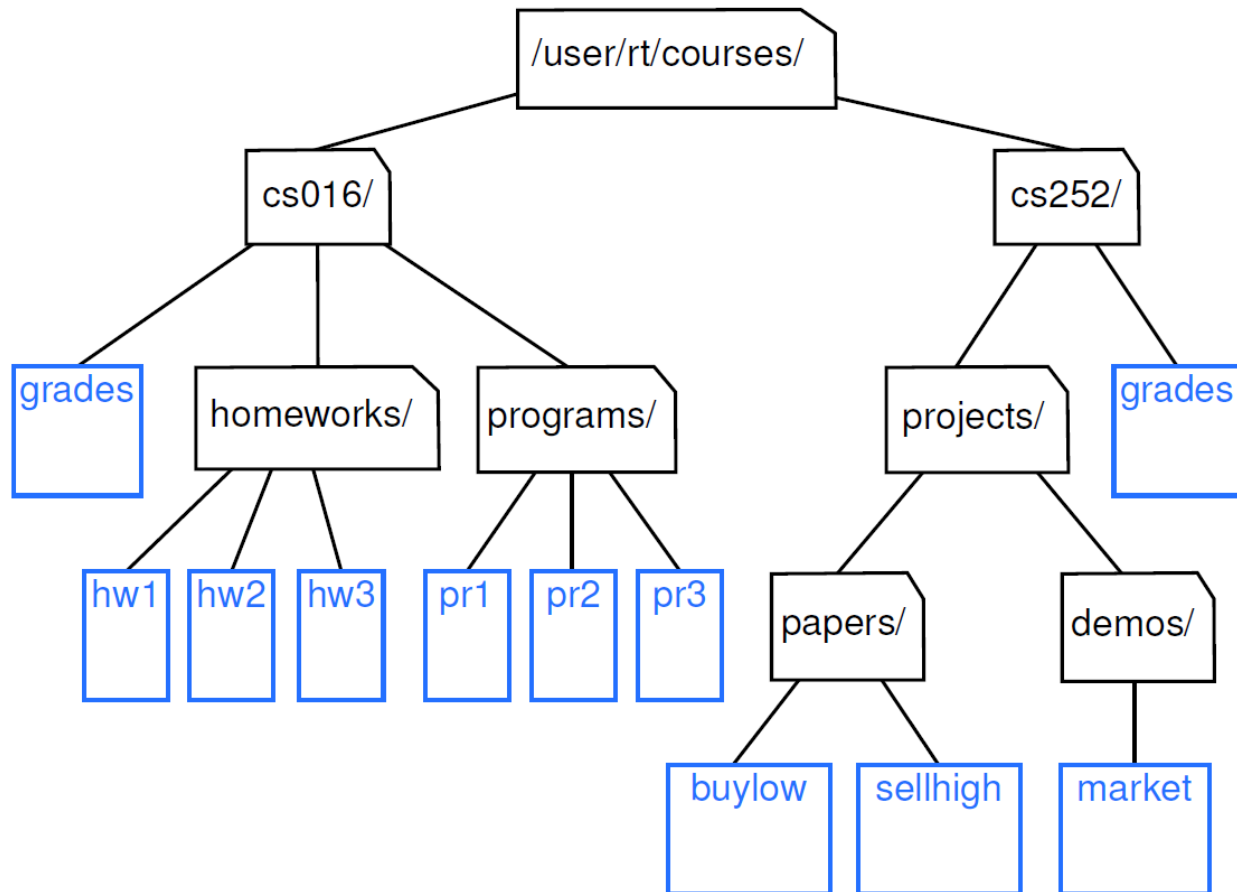


What is a Tree

- ❑ In computer science, a tree is an abstract model of a hierarchical structure
- ❑ A tree consists of nodes with a parent-child relation
- ❑ Applications:
 - Organization charts
 - File systems
 - Programming environments

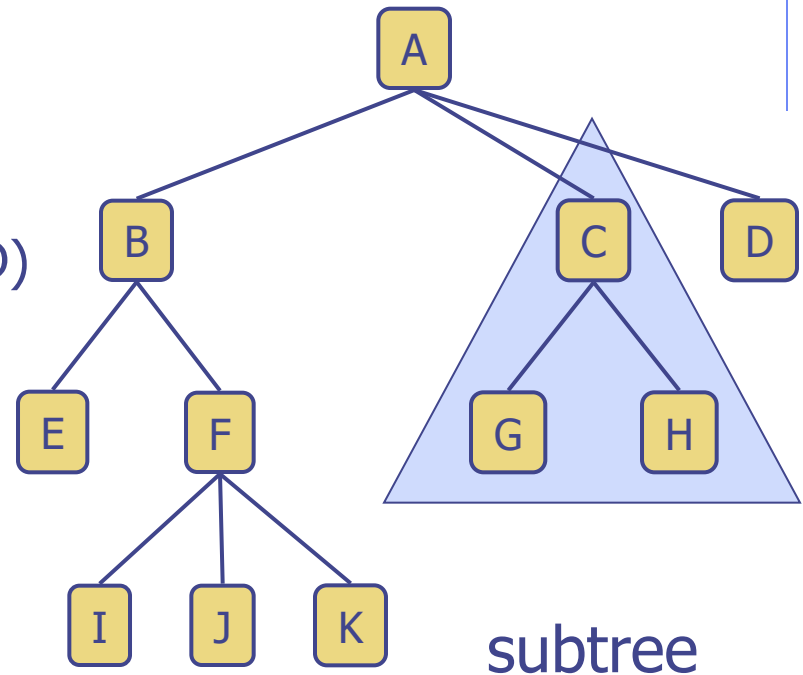


File Systems



Tree Terminology

- ❑ Root: node without parent (A)
- ❑ Internal node: node with at least one child (A, B, C, F)
- ❑ External node (a.k.a. leaf): node without children (E, I, J, K, G, H, D)
- ❑ Ancestors of a node: parent, grandparent, grand-grandparent, etc.
- ❑ Descendant of a node: child, grandchild, grand-grandchild, etc.
- ❑ Subtree: tree consisting of a node and its descendants



Tree ADT

- We use positions to abstract nodes
- Generic methods:
 - Integer `len()`
 - Boolean `is_empty()`
 - Iterator `positions()`
 - Iterator `iter()`
- Accessor methods:
 - position `root()`
 - position `parent(p)`
 - Iterator `children(p)`
 - Integer `num_children(p)`
- ◆ Query methods:
 - Boolean `is_leaf(p)`
 - Boolean `is_root(p)`
- ◆ Update method:
 - element `replace(p, o)`
- ◆ Additional update methods may be defined by data structures implementing the Tree ADT

Abstract Tree Class in Python

```
1 class Tree:
2     """ Abstract base class representing a tree structure. """
3
4     #----- nested Position class -----
5     class Position:
6         """ An abstraction representing the location of a single element. """
7
8         def element(self):
9             """ Return the element stored at this Position. """
10            raise NotImplementedError('must be implemented by subclass')
11
12        def __eq__(self, other):
13            """ Return True if other Position represents the same location. """
14            raise NotImplementedError('must be implemented by subclass')
15
16        def __ne__(self, other):
17            """ Return True if other does not represent the same location. """
18            return not (self == other)          # opposite of __eq__
19
```

Abstract Tree Class in Python

```
20  # ----- abstract methods that concrete subclass must support -----
21  def root(self):
22      """Return Position representing the tree's root (or None if empty)."""
23      raise NotImplementedError('must be implemented by subclass')
24
25  def parent(self, p):
26      """Return Position representing p's parent (or None if p is root)."""
27      raise NotImplementedError('must be implemented by subclass')
28
29  def num_children(self, p):
30      """Return the number of children that Position p has."""
31      raise NotImplementedError('must be implemented by subclass')
32
33  def children(self, p):
34      """Generate an iteration of Positions representing p's children."""
35      raise NotImplementedError('must be implemented by subclass')
36
37  def __len__(self):
38      """Return the total number of elements in the tree."""
39      raise NotImplementedError('must be implemented by subclass')
```

Abstract Tree Class in Python

```
40  # ----- concrete methods implemented in this class -----
41  def is_root(self, p):
42      """Return True if Position p represents the root of the tree."""
43      return self.root( ) == p
44
45  def is_leaf(self, p):
46      """Return True if Position p does not have any children."""
47      return self.num_children(p) == 0
48
49  def is_empty(self):
50      """Return True if the tree is empty."""
51      return len(self) == 0
```


Depth of a Node

- Depth of a node: number of ancestors

```
def depth(self, p):  
    """Return the number of levels separating Position p from the root."""  
    if self.is_root(p):  
        return 0  
    else:  
        return 1 + self.depth(self.parent(p))
```

Height of a Tree

- Height of a tree: maximum depth of any node

```
def _height1(self):                                # works, but  $O(n^2)$  worst-case time
    """Return the height of the tree."""
    return max(self.depth(p) for p in self.positions() if self.is_leaf(p))

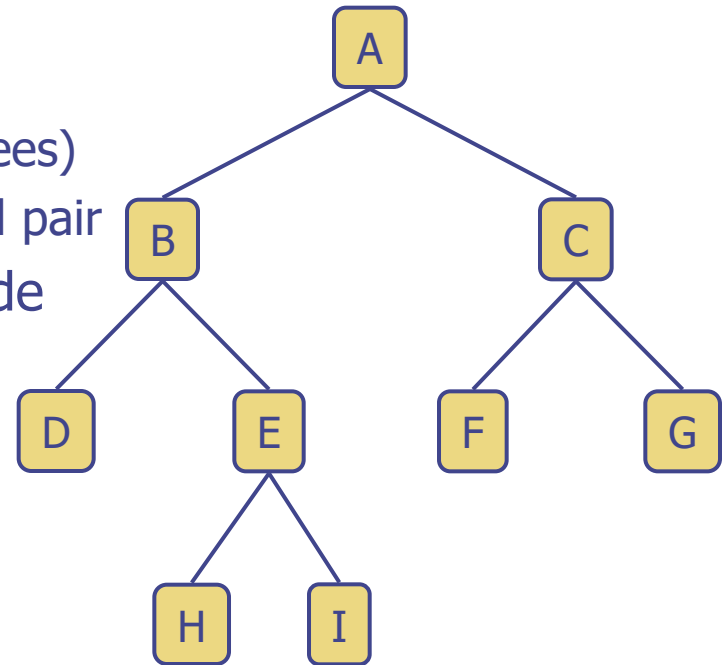
def _height2(self, p):                             # time is linear in size of subtree
    """Return the height of the subtree rooted at Position p."""
    if self.is_leaf(p):
        return 0
    else:
        return 1 + max(self._height2(c) for c in self.children(p))

def height(self, p=None):
    """Return the height of the subtree rooted at Position p.

    If p is None, return the height of the entire tree.
    """
    if p is None:
        p = self.root()
    return self._height2(p)                        # start _height2 recursion
```

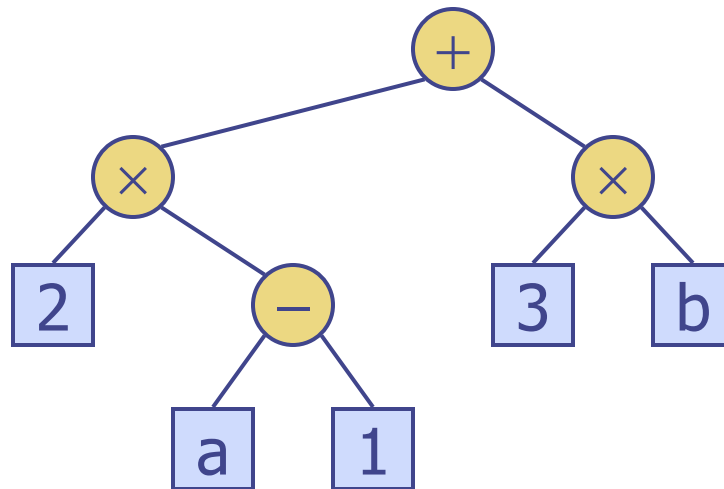
Binary Trees

- ❑ A binary tree is a tree with the following properties:
 - Each internal node has at most two children (exactly two for **full** binary trees)
 - The children of a node are an ordered pair
- ❑ We call the children of an internal node **left child** and **right child**
- ❑ Applications:
 - arithmetic expressions
 - decision processes
 - searching



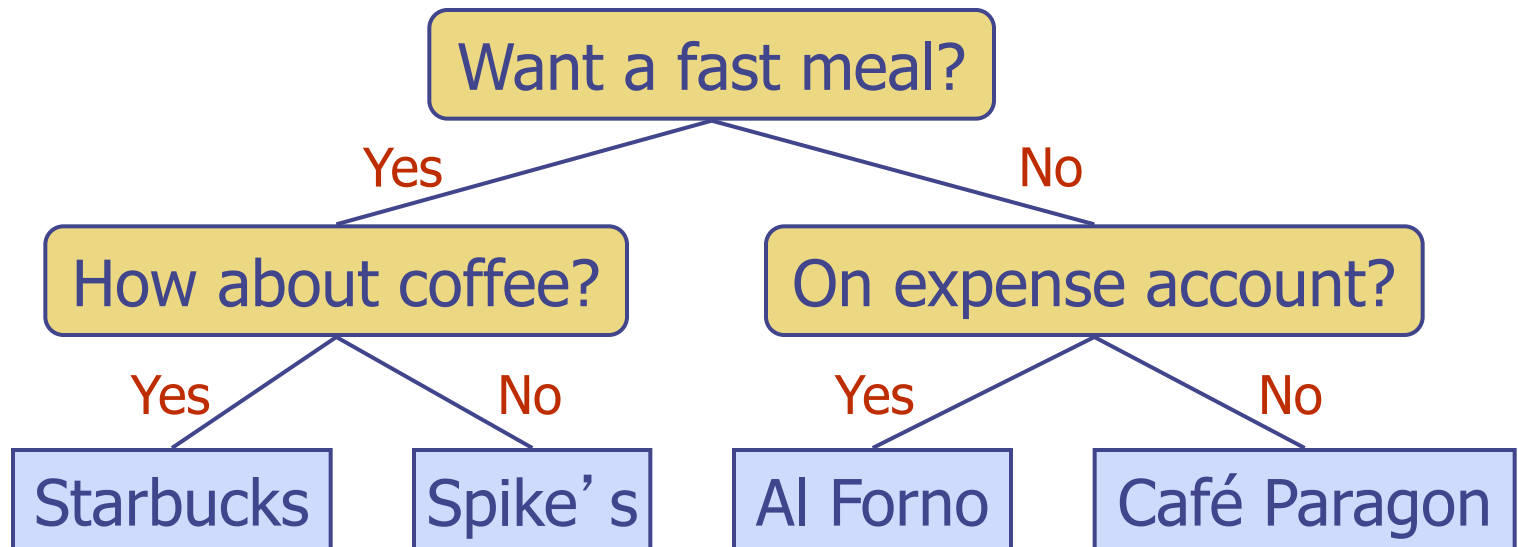
Arithmetic Expression Tree

- Binary tree associated with an arithmetic expression
 - internal nodes: operators
 - external nodes: operands
- Example: arithmetic expression tree for the expression $(2 \times (a - 1) + (3 \times b))$



Decision Tree

- ❑ Binary tree associated with a decision process
 - internal nodes: questions with yes/no answer
 - external nodes: decisions
- ❑ Example: dining decision



BinaryTree ADT

- ❑ The BinaryTree ADT extends the Tree ADT, i.e., it inherits all the methods of the Tree ADT
- ❑ Additional methods:
 - position **left**(p)
 - position **right**(p)
 - position **sibling**(p)
- ❑ Update methods may be defined by data structures implementing the BinaryTree ADT

Binary Tree ADT in Python

```
def sibling(self, p):
    """Return a Position representing p's sibling (or None if no sibling)."""
    parent = self.parent(p)
    if parent is None:
        return None
    else:
        if p == self.left(parent):
            return self.right(parent)
        else:
            return self.left(parent)

def children(self, p):
    """Generate an iteration of Positions representing p's children."""
    if self.left(p) is not None:
        yield self.left(p)
    if self.right(p) is not None:
        yield self.right(p)
```

Properties of Full Binary Trees

□ Notation

n number of nodes

e number of external nodes

i number of internal nodes

h height

◆ Properties:

■ $e = i + 1$

■ $n = 2e - 1$

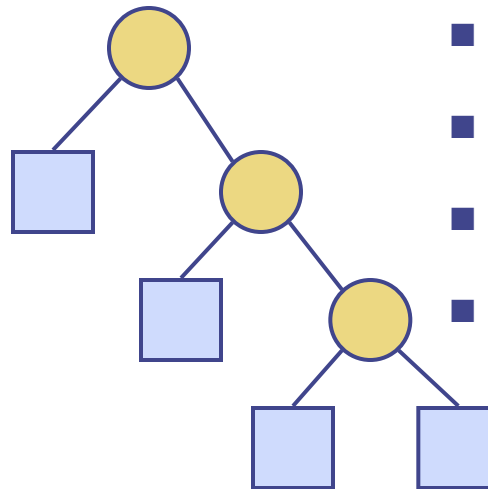
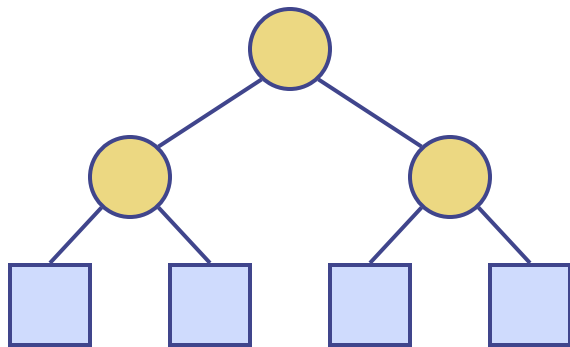
■ $h \leq i$

■ $h \leq (n - 1)/2$

■ $e \leq 2^h$

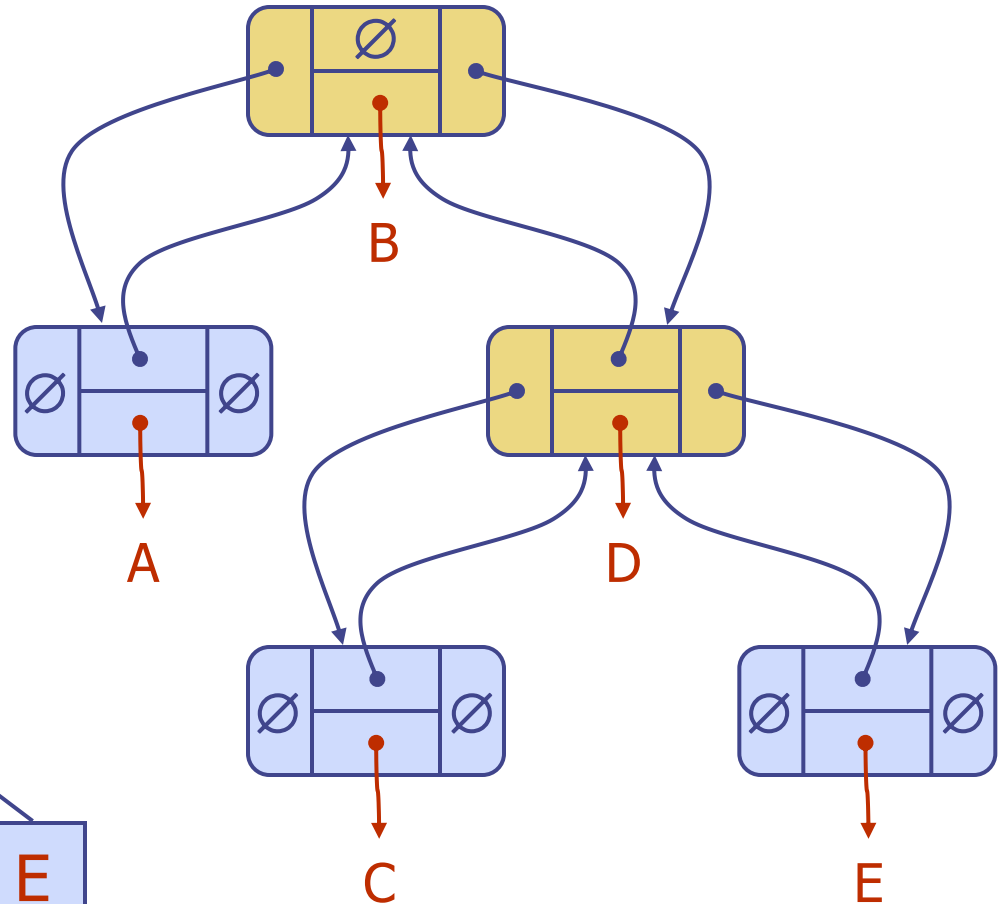
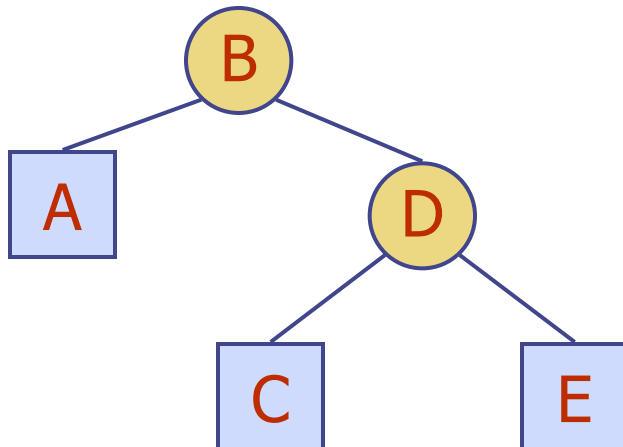
■ $h \geq \log_2 e$

■ $h \geq \log_2 (n + 1) - 1$



Linked Structure for Binary Trees

- A node is represented by an object storing
 - Element
 - Parent node
 - Left child node
 - Right child node



Linked Binary Tree in Python

```
class LinkedBinaryTree(BinaryTree):
    """Linked representation of a binary tree structure."""

    #----- nested _Node class -----
    class _Node:
        """Lightweight, nonpublic class for storing a node."""
        __slots__ = '_element', '_parent', '_left', '_right' # streamline memory usage

        def __init__(self, element, parent=None, left=None, right=None):
            self._element = element
            self._parent = parent
            self._left = left
            self._right = right
```

Linked Binary Tree in Python

```
#----- nonpublic mutators -----  
def _add_root(self, e):  
    """Place element e at the root of an empty tree and return new Position.  
  
    Raise ValueError if tree nonempty.  
    """  
    if self._root is not None:  
        raise ValueError('Root exists')  
    self._size = 1  
    self._root = self._Node(e)  
    return self._make_position(self._root)
```

Linked Binary Tree in Python

```
def _add_left(self, p, e):
    """Create a new left child for Position p, storing element e.

    Return the Position of new node.
    Raise ValueError if Position p is invalid or p already has a left child.
    """
    node = self._validate(p)
    if node._left is not None:
        raise ValueError('Left child exists')
    self._size += 1
    node._left = self._Node(e, node)          # node is its parent
    return self._make_position(node._left)

def _add_right(self, p, e):
    """Create a new right child for Position p, storing element e.

    Return the Position of new node.
    Raise ValueError if Position p is invalid or p already has a right child.
    """
    node = self._validate(p)
    if node._right is not None:
        raise ValueError('Right child exists')
    self._size += 1
    node._right = self._Node(e, node)         # node is its parent
    return self._make_position(node._right)
```

Linked Binary Tree in Python

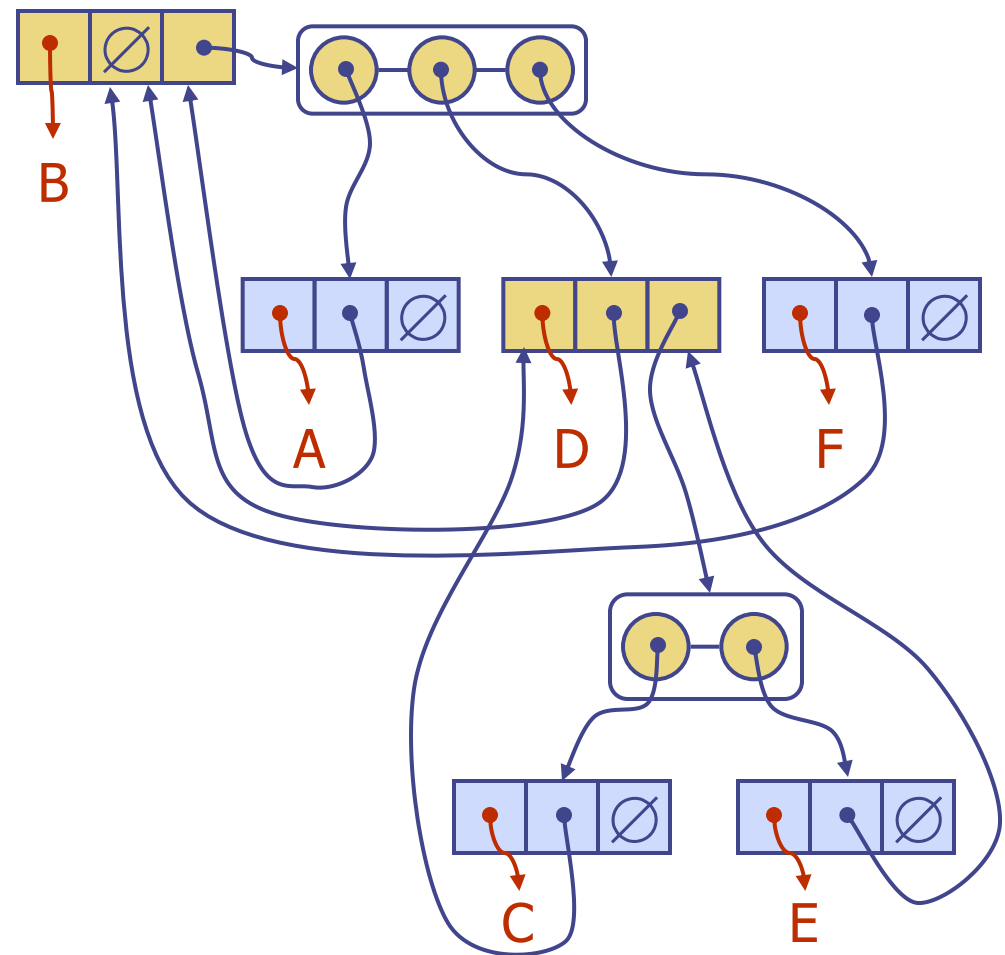
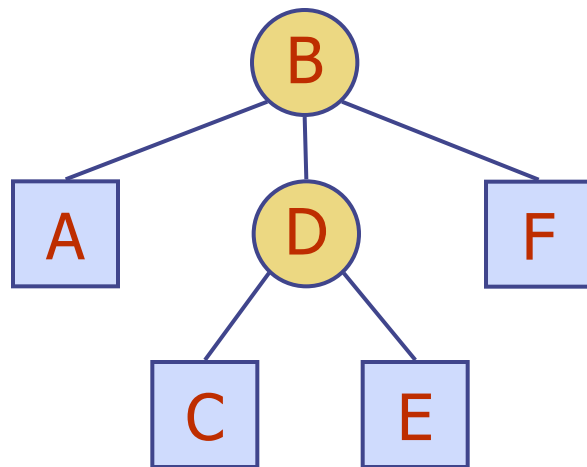
```
def _replace(self, p, e):
    """Replace the element at position p with e, and return old element."""
    node = self._validate(p)
    old = node._element
    node._element = e
    return old

def _delete(self, p):
    """Delete the node at Position p, and replace it with its child, if any.

    Return the element that had been stored at Position p.
    Raise ValueError if Position p is invalid or p has two children.
    """
    node = self._validate(p)
    if self.num_children(p) == 2:
        raise ValueError('Position has two children')
    child = node._left if node._left else node._right # might be None
    if child is not None:
        child._parent = node._parent # child's grandparent becomes parent
    if node is self._root:
        self._root = child # child becomes root
    else:
        parent = node._parent
        if node is parent._left:
            parent._left = child
        else:
            parent._right = child
    self._size -= 1
    node._parent = node # convention for deprecated node
    return node._element
```

Linked Structure for Trees

- A node is represented by an object storing
 - Element
 - Parent node
 - Sequence of children nodes

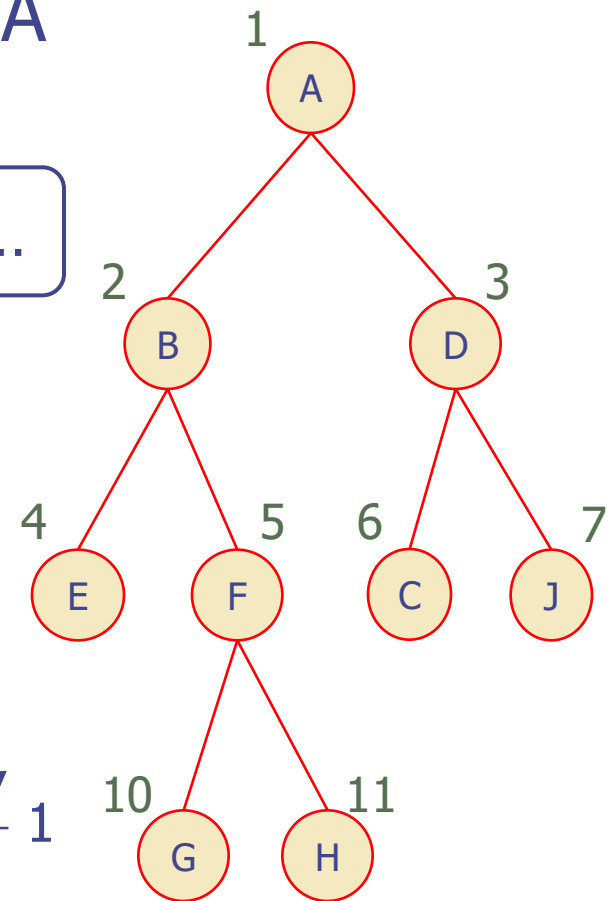


Array-Based Representation of Binary Trees

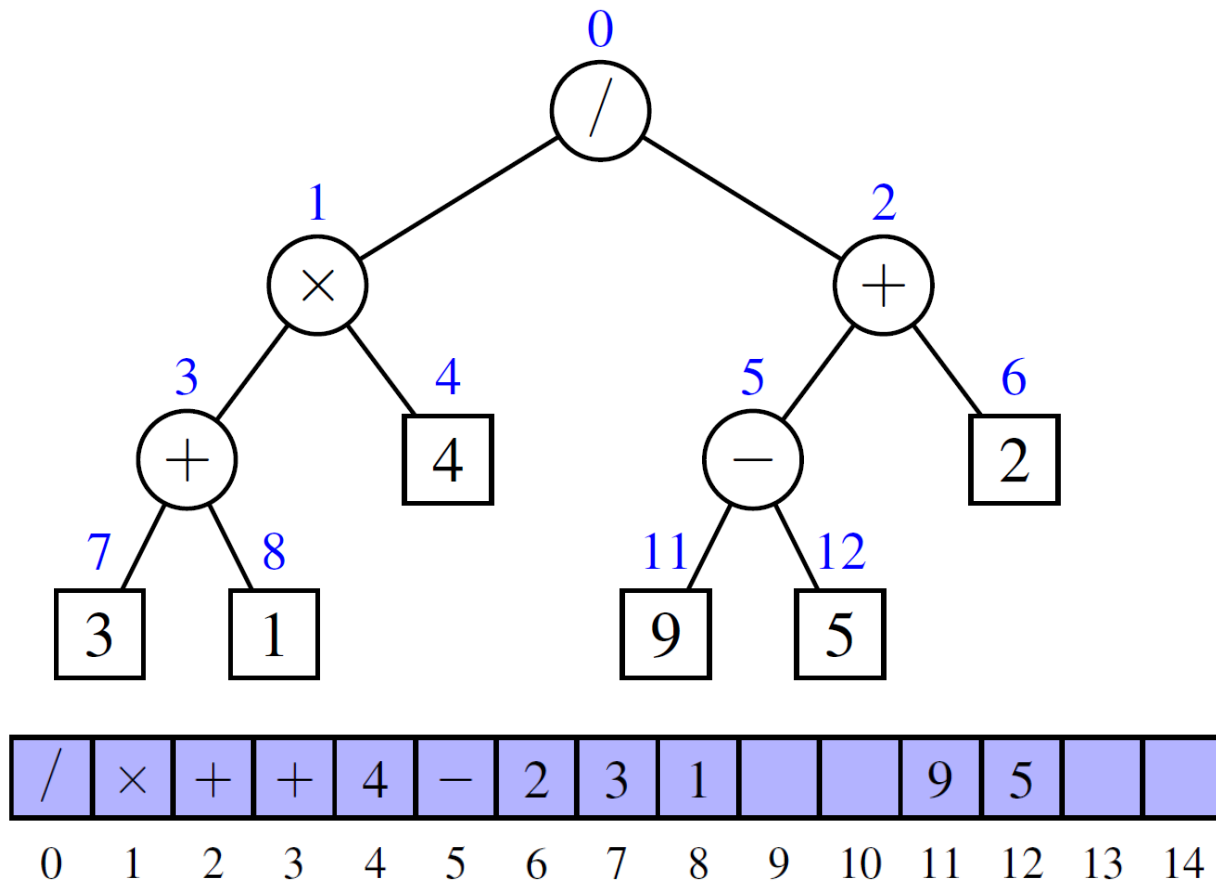
- Nodes are stored in an array A



- Node v is stored at $A[\text{rank}(v)]$
 - $\text{rank}(\text{root}) = 1$
 - if node is the left child of $\text{parent}(\text{node})$,
 $\text{rank}(\text{node}) = 2 \cdot \text{rank}(\text{parent}(\text{node}))$
 - if node is the right child of $\text{parent}(\text{node})$,
 $\text{rank}(\text{node}) = 2 \cdot \text{rank}(\text{parent}(\text{node})) + 1$



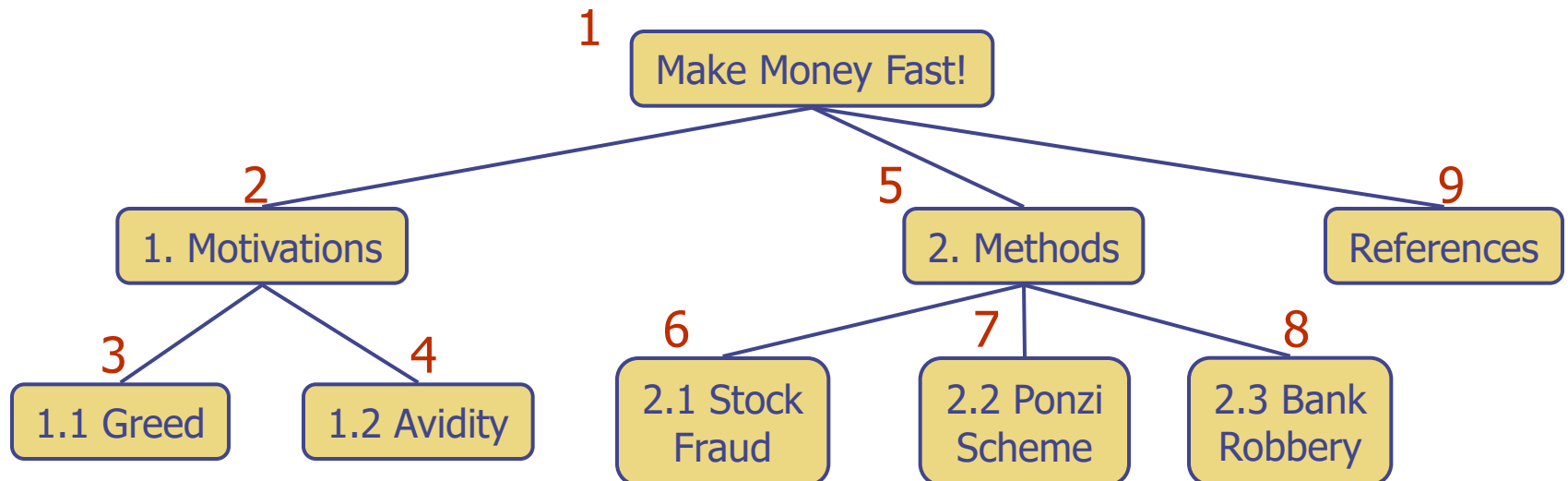
Array-Based Binary Tree



Preorder Traversal

- ❑ A traversal visits the nodes of a tree in a systematic manner
- ❑ In a preorder traversal, a node is visited before its descendants
- ❑ Application: print a structured document

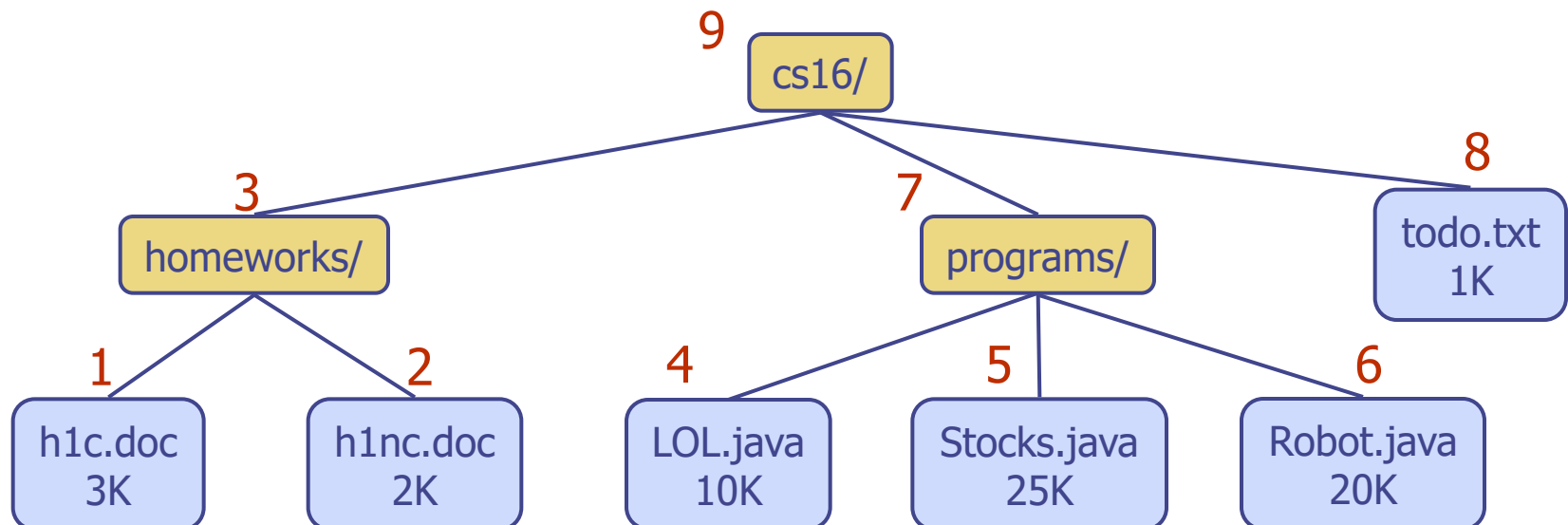
Algorithm *preOrder(v)*
visit(v)
for each child *w* of *v*
preorder (w)



Postorder Traversal

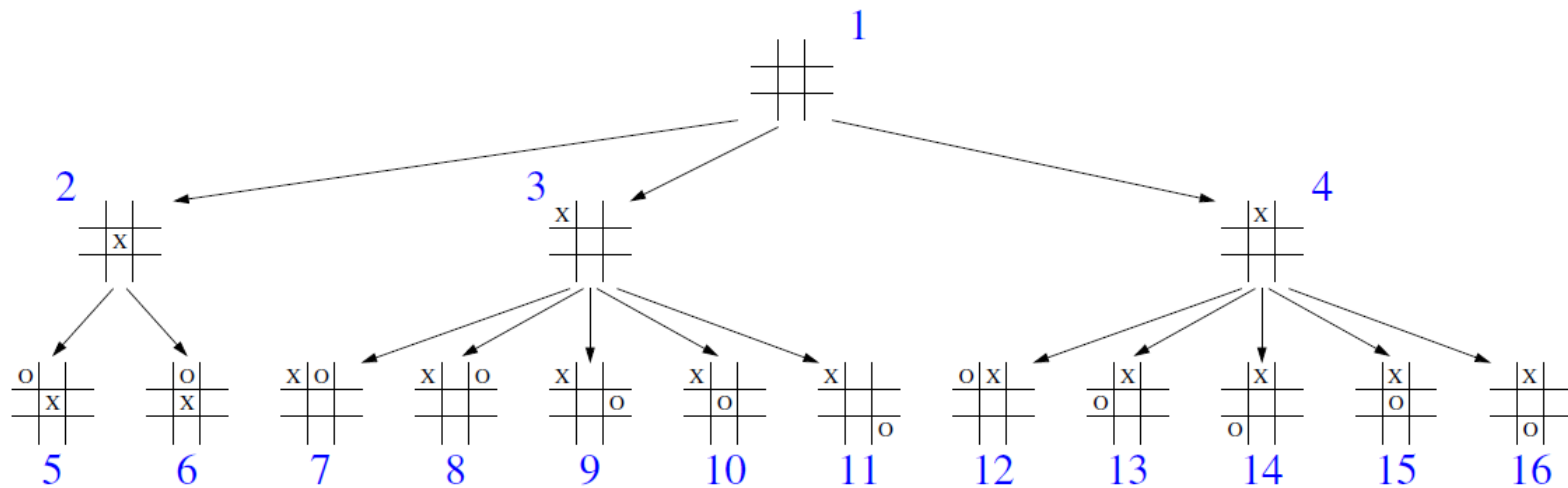
- In a postorder traversal, a node is visited after its descendants
- Application: compute space used by files in a directory and its subdirectories

Algorithm *postOrder(v)*
for each child *w* of *v*
 postOrder(w)
visit(v)



Breadth-First Tree Traversal

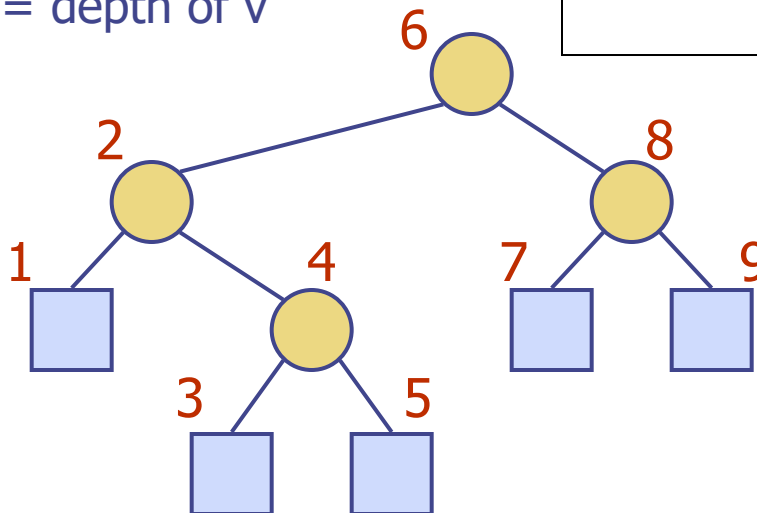
- Visit all the positions at depth d before we visit the positions at depth $d+1$



Inorder Traversal

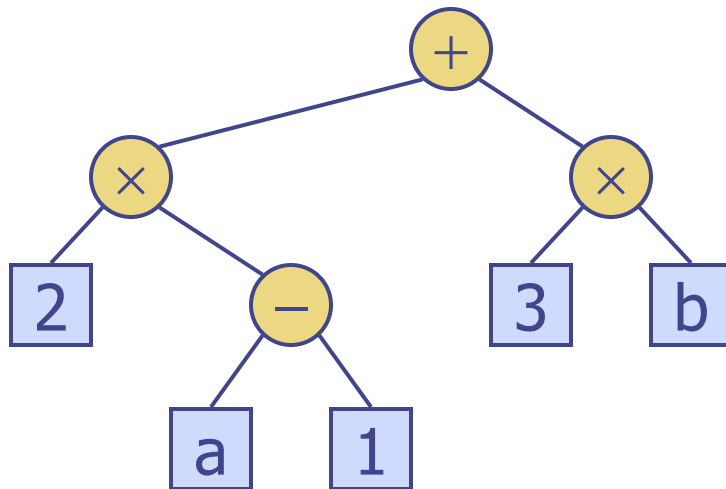
- In an inorder traversal a node is visited after its left subtree and before its right subtree
- Application: draw a binary tree
 - $x(v)$ = inorder rank of v
 - $y(v)$ = depth of v

```
Algorithm inOrder( $v$ )  
  if  $v$  has a left child  
    inOrder (left ( $v$ ))  
  visit( $v$ )  
  if  $v$  has a right child  
    inOrder (right ( $v$ ))
```



Print Arithmetic Expressions

- Specialization of an inorder traversal
 - print operand or operator when visiting node
 - print “(“ before traversing left subtree
 - print “)” after traversing right subtree



Algorithm *printExpression(v)*

if *v* **has a left child**

print("(")

inOrder (*left*(*v*))

print(*v.element* ())

if *v* **has a right child**

inOrder (*right*(*v*))

print (") ")

$((2 \times (a - 1)) + (3 \times b))$

Evaluate Arithmetic Expressions

- Specialization of a postorder traversal
 - recursive method returning the value of a subtree
 - when visiting an internal node, combine the values of the subtrees

Algorithm *evalExpr(v)*

if *is_leaf*(v)

return *v.element* ()

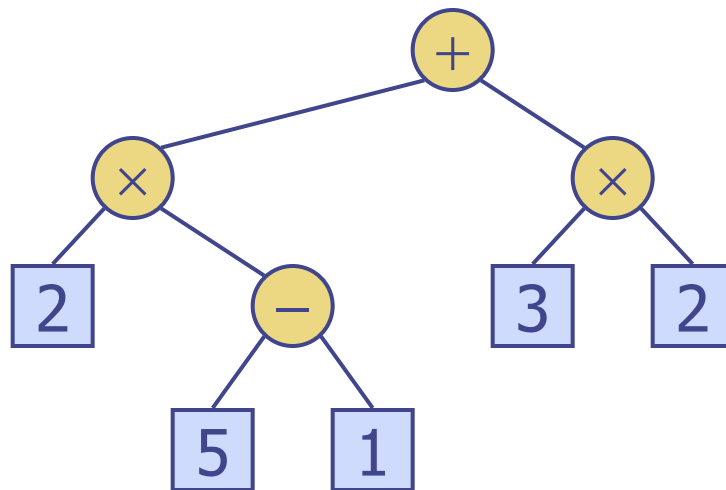
else

x $\leftarrow$ *evalExpr*(*left* (v))

y $\leftarrow$ *evalExpr*(*right* (v))

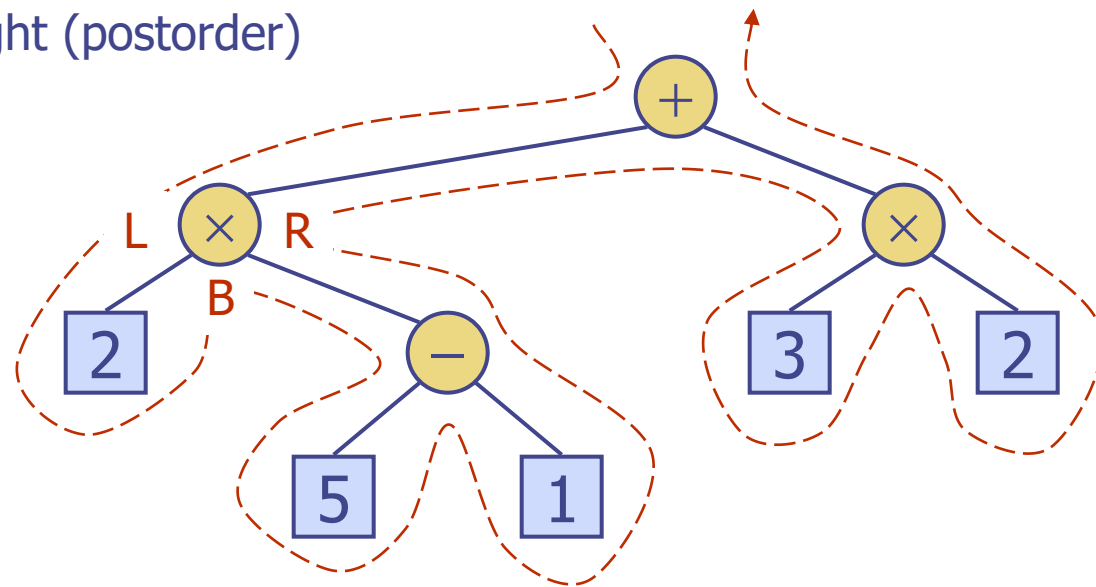
$\diamond \leftarrow$ operator stored at v

return *x* $\diamond$ *y*



Euler Tour Traversal

- ❑ Generic traversal of a binary tree
- ❑ Includes a special cases the preorder, postorder and inorder traversals
- ❑ Walk around the tree and visit each node three times:
 - on the left (preorder)
 - from below (inorder)
 - on the right (postorder)



Euler Tour Traversal in Python

```
class EulerTour:
    """Abstract base class for performing Euler tour of a tree.

    _hook_previsit and _hook_postvisit may be overridden by subclasses.
    """
    def __init__(self, tree):
        """Prepare an Euler tour template for given tree."""
        self._tree = tree

    def tree(self):
        """Return reference to the tree being traversed."""
        return self._tree

    def execute(self):
        """Perform the tour and return any result from post visit of root."""
        if len(self._tree) > 0:
            return self._tour(self._tree.root(), 0, [])    # start the recursion
```


Euler Tour Traversal in Python

```
def _tour(self, p, d, path):
    """Perform tour of subtree rooted at Position p.

    p          Position of current node being visited
    d          depth of p in the tree
    path       list of indices of children on path from root to p
    """
    self._hook_previsit(p, d, path)                # "pre visit" p
    results = []
    path.append(0)                                # add new index to end of path before recursion
    for c in self._tree.children(p):
        results.append(self._tour(c, d+1, path))    # recur on child's subtree
        path[-1] += 1                               # increment index
    path.pop()                                     # remove extraneous index from end of path
    answer = self._hook_postvisit(p, d, path, results) # "post visit" p
    return answer
```

Euler Tour Traversal in Python

```
def _hook_previsit(self, p, d, path):
    """Visit Position p, before the tour of its children.

    p          Position of current position being visited
    d          depth of p in the tree
    path       list of indices of children on path from root to p
    """
    pass

def _hook_postvisit(self, p, d, path, results):
    """Visit Position p, after the tour of its children.

    p          Position of current position being visited
    d          depth of p in the tree
    path       list of indices of children on path from root to p
    results    is a list of values returned by _hook_postvisit(c)
               for each child c of p.
    """
    pass
```

ParantesizeTour

```
class ParenthesizeTour(EulerTour):
    def _hook_previsit(self, p, d, path):
        if path and path[-1] > 0:           # p follows a sibling
            print(', ', end='')              # so preface with comma
        print(p.element(), end='')          # then print element
        if not self.tree().is_leaf(p):      # if p has children
            print(' (', end='')             # print opening parenthesis

    def _hook_postvisit(self, p, d, path, results):
        if not self.tree().is_leaf(p):      # if p has children
            print(')', end='')               # print closing parenthesis
```

DiskSpaceTour

```
class DiskSpaceTour(EulerTour):
    def _hook_postvisit(self, p, d, path, results):
        # we simply add space associated with p to that of its subtrees
        return p.element().space() + sum(results)
```